Full Stack Development Project Documentation

RentEase Platform

1. Introduction

- **Project Title:** RentEase: Comprehensive House Rental & Booking Marketplace
- **Team Members:** Shaan Bethala (Full Stack Developer & Architecture Lead)

2. Project Overview

- **Purpose:** The project aims to bridge the gap between prospective tenants (Renters), Property Owners, and Administrators by modernizing real estate discovery, strictly verifying landlord identities, and coordinating physical tour bookings without manual SMS coordination.
- **Features:**
  - Role-Based Access Control (Renter, Owner, Admin)
  - Admin Verification Gateway to prevent scam listings
  - Dynamic Explorer Grid for property discovery
  - Inline "Lease Scheduler" for 1-click tour approvals and cancellations

3. Architecture

- **Frontend:** Developed as a Single Page Application (SPA) using React 18 (Vite) and styled with Tailwind CSS to establish a distinctive Off-White & Deep Charcoal aesthetic.
- **Backend:** Built on a unified Node.js and Express.js REST API layer that handles business logic, authorization middleware, and unified port mapping (port 3000).
- **Database:** Utilizes a lightweight, transactional JSON database (`db.json`) modeling relational entities across Users, Properties, and Bookings.

4. Setup Instructions

- **Prerequisites:** Node.js (v18 or higher), npm (Node Package Manager), and Git.
- **Installation:**
  - Clone the repository: `git clone <repo-url>`
  - Navigate to the project root and install backend dependencies: `npm install`
  - Navigate to the `/client` directory and install frontend dependencies: `npm install`
  - Configure environment variables (.env) for JWT Secrets.

5. Folder Structure

- **Client:** Contains the React frontend (components, pages, assets, and Tailwind configurations).
- **Server:** Contains the Node.js backend (Express routes, authentication controllers, middleware, and the `db.json` data store).

6. Running the Application

- Provide commands to start the frontend and backend servers locally.
  - **Frontend:** Run `npm run dev` in the client directory.
  - **Backend:** Run `npm start` in the server directory.
  - *Note:* In production, the unified server pipeline serves built static assets directly via Express.

7. API Documentation

- **Authentication:** `POST /api/auth/login`, `POST /api/auth/register`
- **Properties:** `GET /api/properties` (Fetch all), `POST /api/properties` (Create listing)
- **Bookings:** `GET /api/bookings`, `PUT /api/bookings/:id/status`
- Request parameters typically require a valid Bearer token for private routes. Responses are standardized JSON payloads containing status codes and data objects.

8. Authentication

- Authentication and role-based authorization are managed securely using JSON Web Tokens (JWT).
- Upon successful login, a cryptographically signed JWT payload (containing userId, email, and role type) is generated and passed to the client. The client attaches this token to the `Authorization: Bearer <token>` header for subsequent protected requests. Express middleware validates this token before granting access.

9. User Interface

- The UI is divided into distinct role-based workspaces (Renter Dashboard, Owner Dashboard, Admin Portal).
- Key features include a visual housing gallery (Dynamic Explorer Grid), a Listing Composer modal, and visual Status Logs tracking active, confirmed, and cancelled tour requests with clear timeline indicators.

10. Testing

- Manual API testing utilizing Postman to verify route functionality and JWT validation.
- User Acceptance Testing (UAT) focused on core user flows: Client Application responsiveness, Authentication, Property Discovery, Tour Booking Coordinator, and Database Transactions.

11. Screenshots or Demo

- A live demonstration of the application and corresponding UI screenshots can be found in the project's root `/docs/assets` folder or via the provided GitHub Pages deployment link.

12. Known Issues

- The lightweight JSON database (`db.json`) is excellent for rapid prototyping but lacks concurrency safety for high-volume write operations.
- Manual geographical inputs do not currently sync with an external live map API.

13. Future Enhancements

- Migration of the data layer from `db.json` to a scalable NoSQL/RDBMS database (e.g., MongoDB or PostgreSQL).
- Integration of a Map API (like Google Maps) for 1-click property directions.
- Implementation of in-app digital lease signing and automated SMS notifications for tour confirmations.